

MIND: Microsoft News Dataset

A Large-Scale English News Recommendation Research

Team #6



Fielding
School of Public Health

MIND Dataset Introduction

MIND is Microsoft's large-scale English news recommendation benchmark. This deck introduces what the dataset is, how it is organized, and why it is widely used in recommender-system research.

160k+

English news articles

15M+

impression logs

1M

users

SOURCE

Official website:
<https://msnews.github.io>

Primary paper:
Wu et al., ACL 2020

WHY IT MATTERS

MIND is a public benchmark for personalized news recommendation. It supports content-based, collaborative, recall-stage, and ranking-stage modeling under a shared evaluation setup.

MAIN IDEA

Each impression contains a candidate set of news articles, plus the user's earlier click history. Models use this context to predict which candidate items are most likely to be clicked.

What Is MIND?

MIND stands for Microsoft News Dataset and is designed for news recommendation research.

OFFICIAL DESCRIPTION

The MIND website describes it as a large-scale English dataset for news recommendation research, collected from anonymized Microsoft News behavior logs and released to support benchmark-driven work in recommendation.

RESEARCH VALUE

Because MIND includes both rich article text and interaction behavior, it supports a wide range of methods: content-based models, collaborative filtering, neural news recommenders, and two-stage recall-and-ranking systems.

CORE DATA MODALITIES

News content:

- title
- abstract
- body
- category
- entities

Behavior logs:

- clicks
- non-clicked impressions
- user click history before each impression

MIND: Microsoft News Dataset

A Large-Scale English Dataset for News Recommendation Research

Dataset Scale And Versions

The website highlights both the full dataset and a smaller quick-start variant.

160k+

news articles

15M+

impression logs

1M

users

50k

users in MIND-small

MIND-LARGE

The full dataset is intended for benchmark-scale training, validation, and leaderboard submission. It is the version typically used when comparing against published state-of-the-art systems.

MIND-SMALL

The website also provides MIND-small, created by randomly sampling 50,000 users and their behavior logs. This version is especially useful for quick experiments, tutorials, and reproducible classroom projects.

DOWNLOAD SPLITS

The site provides training, validation, and test sets. It also links to Microsoft Azure Open Datasets for easier cloud access to the large-scale version.

PRACTICAL TAKEAWAY

For learning and baseline building, MIND-small is usually the easiest place to start. For serious model comparison and leaderboard-style benchmarking, the full MIND release is the more representative choice.

We use the MIND-small dataset due to limited computational resources.

Main Files And Data Structure

MIND combines article metadata with user behavior logs.

news.tsv

One row per news article.

Common fields:

- title
- abstract
- body
- category
- entities

behaviors.tsv

One row per impression event.

Contains:

- user ID
- timestamp
- clicked history before the impression
- candidate impressions with click / non-click labels

This file supports interaction modeling and ranking evaluation.

This file supports content representation and news-side feature engineering.

HOW THEY WORK TOGETHER

Most pipelines merge article features from news.tsv with candidate rows expanded from behaviors.tsv. That creates the candidate-level training table used by recall and ranking models.

WHY THIS IS USEFUL

This structure is flexible: text-based models can focus on article content, collaborative methods can focus on behavior, and supervised CTR/ranking models can combine both.

Repository Map And Evaluation Workflow

Every notebook and saved artifact is represented in the reporting pipeline.

NOTEBOOKS

round1_descriptive_stats.ipynb
popularity_baseline.ipynb
tfidf.ipynb
itemcf.ipynb
lr_ctr.ipynb
lightgbm_ctr.ipynb

OUTPUTS

round1_descriptive_stats_output/
popularity_baseline_output/
tfidf_output/
itemcf_output/
lr_ctr_output/
lightgbm_ctr_output/

COMMON PROTOCOL

1. Expand impressions to candidate-level rows.
2. Score each candidate within an impression.
3. Export prediction.txt or metrics files.
4. Evaluate AUC, MRR, nDCG@5, nDCG@10 on validation.

WHY THIS MATTERS

We moves from simple heuristics to stronger CTR models while keeping the same evaluation target. That makes the final comparison fair and easy to explain in one deck.

PROJECT HEADLINE

We mixes two baseline families: recall-style methods that generate or pre-score candidates from popularity/content/collaborative signals and ranking-style methods that learn click-oriented ordering over candidate-level rows.

Recall vs Ranking

For a full recommender system, these two stages play different roles and usually need separate baselines.

RECALL / RETRIEVAL

Goal: quickly surface a smaller candidate pool from the large corpus.

Typical signals:

- global popularity
- content similarity
- collaborative filtering

In this repo:

Popularity, TF-IDF, and ItemCF fit this family best.

RANKING

Goal: reorder an existing candidate set so clicked items move above non-clicked items.

Typical signals:

- candidate position
- user-history match features
- metadata features
- supervised CTR labels

In this repo:

Logistic Regression CTR and LightGBM CTR are ranking baselines.

Dataset Overview

The train/validation split is stable, sparse, and large enough for meaningful baseline comparison.

5.84M

train candidates

2.74M

valid candidates

37.23

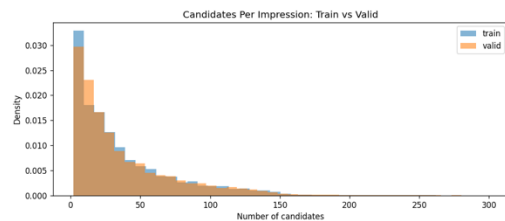
avg candidates/train
impression

4.06%

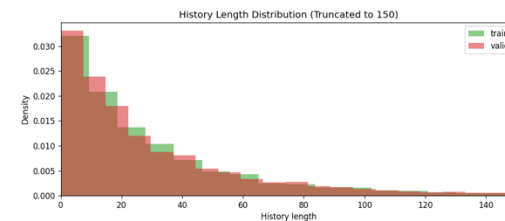
validation CTR

Key descriptive takeaways

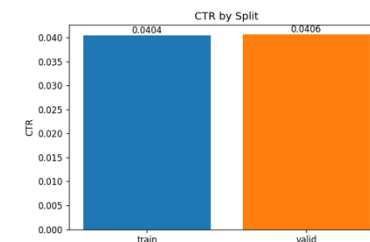
- Train and validation candidate density are almost identical.
- CTR stays near 4%, confirming a sparse ranking task.
- History length is long-tailed across users.
- News and sports dominate the category mix.



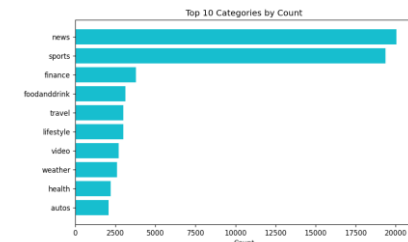
Candidates per impression: train vs valid



History length: train vs valid



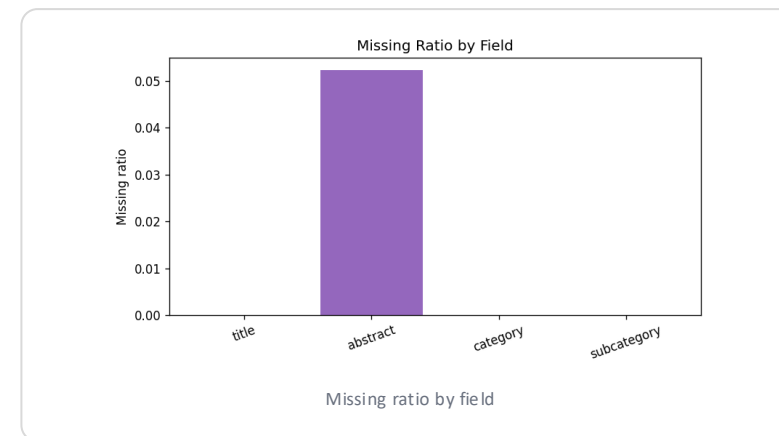
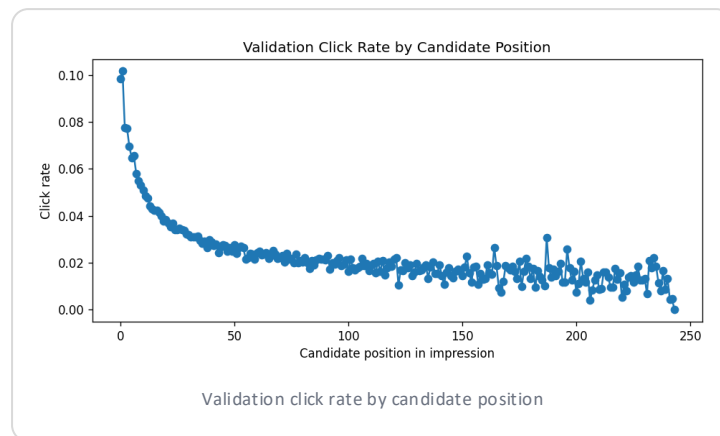
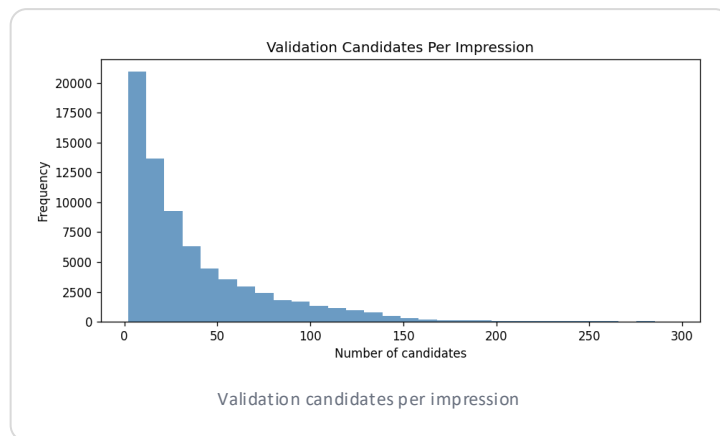
CTR by split



Top-10 category count

Descriptive Findings And Modeling Implications

The saved screenshots highlight where simple baselines help and where richer features are needed.



INTERPRETATION

Candidate count and history-length distributions overlap strongly across splits, so comparison risk from distribution shift is low.

The click-rate-by-position chart suggests presentation order still carries signal.

Missingness is concentrated in abstract, which motivates fallback handling and lightweight text statistics in later models.

MODELING CONSEQUENCE

These plots justify the design of later baselines:

- popularity for a quick prior
- TF-IDF for content similarity
- history-match features for CTR models
- candidate position as a useful context feature

Popularity Baseline

The simplest reference model ranks validation candidates by global training click counts.

popularity_baseline.ipynb

RECALL-STYLE BASELINE

METHOD

1. Parse impressions from behaviors.tsv.
2. Count how often each news article was clicked in training.
3. Score each candidate by that global popularity.
4. Evaluate ranking metrics on validation.

WHY KEEP IT

This notebook is the minimum sanity-check baseline for the end-to-end ranking pipeline. It is cheap, transparent, and useful for verifying that stronger methods are learning beyond a popularity prior.

0.5318

AUC

0.2671

MRR

0.2460

nDCG@5

0.3098

nDCG@10

SUMMARY

Popularity beats random ranking and gives a consistent baseline, but it cannot personalize by user history or article content. The next notebooks improve on this with content, collaborative, and CTR-specific signals.

TF-IDF Content Baseline

This notebook turns title and abstract text into a user profile and scores each candidate by cosine similarity.

tfidf.ipynb

RECALL-STYLE BASELINE

PIPELINE

- Build text with title + abstract.
- Fit a TF-IDF vectorizer over the news corpus.
- Average clicked-history vectors into a user representation.
- Score candidates by cosine similarity to that profile.

LIMITATIONS

No semantic understanding, no collaborative signal, and cold-start users with empty history get score 0. Even so, this notebook offers a strong content-based reference and a reusable text-processing path.

0.5656

AUC

0.3154

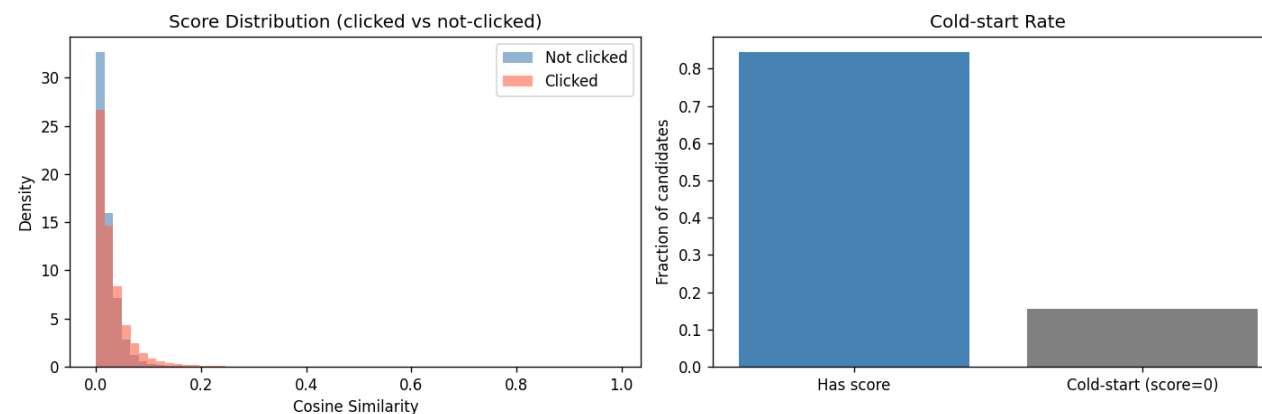
MRR

0.2882

nDCG@5

0.3513

nDCG@10



Score distribution of clicked vs. non-clicked candidates

Item-Based Collaborative Filtering

ItemCF scores each validation candidate by its similarity to the items in a user's click history.

itemcf.ipynb

RECALL-STYLE BASELINE

CORE IDEA

Build train-time user-clicked item sets, then compare a candidate item with the user's history through pairwise similarity. In this notebook, the similarity function is Jaccard-based and fully collaborative.

WORKFLOW

clicked history
↓
find similar items in train clicks
↓
aggregate similarity score per candidate
↓
rank candidates inside each impression

0.5212

AUC

0.2634

MRR

0.2406

nDCG@5

0.3039

nDCG@10

INTERPRETATION

ItemCF provides a pure collaborative baseline, but its AUC and ranking metrics are weaker than the content and CTR models here. That suggests user-history overlap alone is not enough in this Round 1 setup.

Logistic Regression CTR

A lightweight tabular CTR model adds explicit history-match, context, and metadata features.

[lr_ctr.ipynb](#)

RANKING BASELINE

Task Formulation

Logistic Regression estimates click probability for each (user, candidate news) pair.

User history, news content, and **contextual signals** are encoded as a feature vector;

Model learns a linear decision boundary to score each candidate. Final ranking is by **predicted click probability**.

 **Efficient Training**

 **Interpretable**

 **Great Baseline**

Feature Groups

User History

- history_len
- history_unique_len

History-Match

- same_category_count / ratio
- same_subcategory_count / ratio

Context

- hour
- weekday
- is_weekend

Candidate News

- category / subcategory (OHE)
- title & abstract word/char count

Pipeline



Logistic Regression Results & Findings

These details come from the separate lr_ctr_baseline.pptx and strengthen the LR interpretation section.

lr_ctr.ipynb

RANKING BASELINE

0.6112

AUC

0.3130

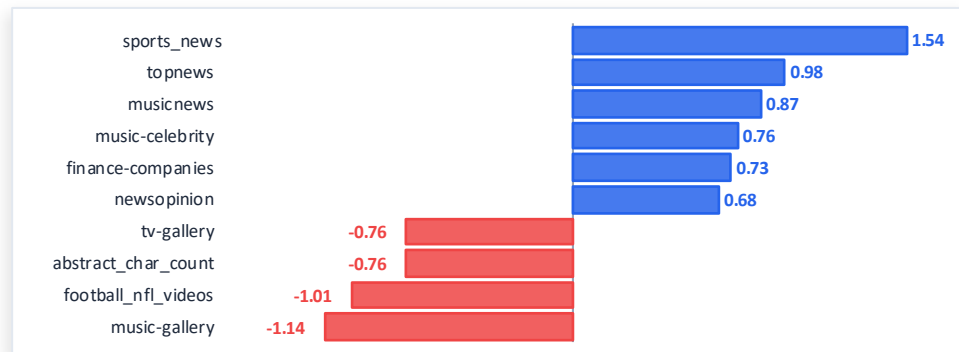
MRR

0.3005

nDCG@5

0.3640

nDCG@10



KEY FINDINGS

Simple tabular features already capture effective click signals: LR reaches AUC 0.6112 without text embeddings.

Subcategory features dominate the linear model, with sports_news and topnews as strong positives while music-gallery and football_nfl_videos are strong negatives.

TOP POSITIVE COEFFICIENTS

sports_news +1.54
topnews +0.98
musicnews +0.87
music-celebrity +0.76
finance-companies +0.73
newsopinion +0.68

TOP NEGATIVE COEFFICIENTS

tv-gallery -0.76
abstract_char_count -0.76
football_nfl_videos -1.01
music-gallery -1.14

TUNING TAKEAWAY

The separate LR baseline slide deck reports that tuning around C=0.1 only produced marginal gains. The bottleneck is featuring expressiveness rather than regularization strength.

LightGBM CTR

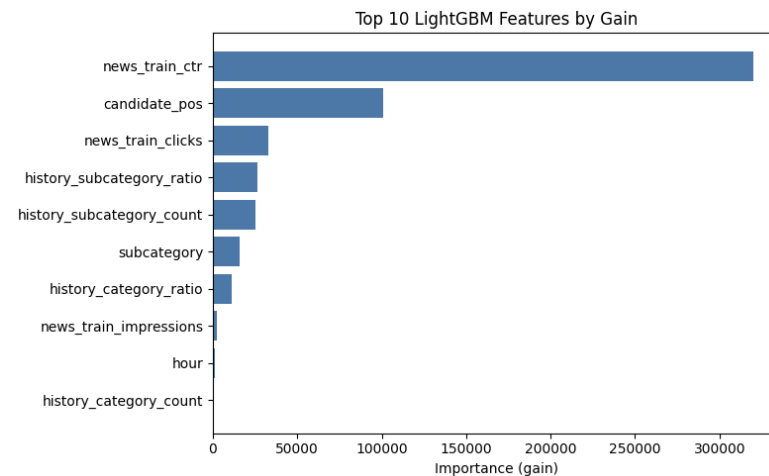
The strongest ranking baseline adds nonlinear tree interactions and train-time popularity statistics.

lightgbm_ctr.ipynb

RANKING BASELINE

FEATURES

candidate position, history length, unique history length, hour / weekday / weekend, category / subcategory, title / abstract lengths, history-category ratios, and train-time news popularity statistics



0.6110

AUC

0.3332

MRR

0.3141

nDCG@5

0.3716

nDCG@10x

TOP IMPORTANCE

news_train_ctr captures item-level global popularity learned from training clicks, while candidate_pos captures exposure/order effects in the candidate list. In contrast, user-interest matching features (history_category/subcategory count/ratio and category fields) encode personalization and contribute less gain. This indicates the model is currently driven more by popularity and position priors than by deep user-specific preference modeling.

TUNED PARAMS

n_estimators=400
learning_rate=0.1
num_leaves=31
max_depth=6
feature_fraction=0.8
bagging_fraction=0.7

TAKEAWAY

LightGBM is effectively tied with Logistic Regression on AUC and wins the ranking-oriented metrics MRR, nDCG@5, and nDCG@10. That makes it the best overall Round 1 baseline for recommendation quality.

Popularity over-reliance risk: The model is heavily driven by news_train_ctr, so it may over-prioritize already popular items and under-serve long-tail or emerging content.

Position-bias amplification risk: Strong dependence on candidate_pos suggests the model may learn exposure order effects rather than pure relevance, which can hurt robustness if ranking policies change.

Overall Comparison And Recommendation

The deck closes by comparing all five baselines under the same validation protocol.

Model	AUC	MRR	nDCG@5	nDCG@10
Popularity	0.5318	0.2671	0.2460	0.3098
TF-IDF	0.5656	0.3154	0.2882	0.3513
ItemCF	0.5212	0.2634	0.2406	0.3039
Logistic Regression	0.6112	0.3130	0.3005	0.3640
LightGBM	0.6110	0.3332	0.3141	0.3716

RECOMMENDATION

Use one baseline per stage.

Recall: TF-IDF.
Ranking: LightGBM CTR.

NEXT STEPS

1. Use TF-IDF as the recall-stage baseline.
2. Use LightGBM CTR as the ranking-stage baseline.
3. use a neural model (preferably NRMS, or NAML/LSTUR) to rescore and reorder those candidates.

Q&A

- 1. We're planning to use a more advanced model next. Does this direction make sense, or would you suggest a different priority?**
- 2. For cold-start scenarios, particularly when user interaction data is limited, which approach would you recommend prioritizing? We are planning to build a dashboard or application to demonstrate our model's real-world behavior, and we expect to encounter this issue in practice.**